Universidad EAFIT

Sistemas operativos

Reporte Parcial 2

Alessandro Soccol Mejía

Juan Pablo Parra El-Masri

Juan Antonio Buendía Sánchez

Ismael García Ceballos

Edison Valencia

Abril 07 de 2026

Smart Backup Kernel-Space Utility

1. Introducción:

El objetivo de este proyecto es implementar un sistema de respaldo de archivos que permita comparar el rendimiento entre dos métodos de copia de archivos: uno basado en llamadas al sistema (syscalls) que simulan comportamiento en espacio de kernel, y otro basado en las librerías estándar de C que operan en espacio de usuario. Este ejercicio permite comprender conceptos fundamentales de sistemas operativos como manejo de archivos, buffers, cambio de contexto y diferencias entre kernel space y user space.

2. Objetivo:

Desarrollar una herramienta de respaldo (backup) que permita copiar archivos utilizando dos enfoques distintos: uno de bajo nivel mediante llamadas al sistema (`open`, `read, write`) y otro utilizando las funciones de alto nivel de la librería estándar de C (`fopen, fread, fwrite`). Posteriormente se realiza una comparación del rendimiento entre ambos métodos.

3. Arquitectura del proyecto:

El proyecto está dividido en tres componentes principales:

- `smart_copy.h`: Define las funciones, constantes y estructuras necesarias para el funcionamiento del sistema.
- `backup_engine.c`: Implementa la lógica principal de copia de archivos tanto usando llamadas al sistema como librerías estándar.
- `main.c`: Proporciona la interfaz de usuario del programa y realiza la comparación de rendimiento entre ambos métodos.

4. Implementación de la Función tipo Syscall:

La función principal del proyecto simula una llamada al sistema llamada `sys_smart_copy`. Esta función utiliza descriptores de archivo y operaciones de bajo nivel como `open(), read() y write()` para copiar archivos. El uso de estas funciones permite interactuar directamente con el sistema operativo, simulando el comportamiento de una función en espacio de kernel.

5. Uso de Buffer de 4096 Bytes:

Para optimizar la copia de archivos, se utiliza un buffer de 4096 bytes. Este tamaño corresponde típicamente al tamaño de página de memoria en muchos sistemas operativos. El uso de buffers reduce la cantidad de llamadas al sistema necesarias, mejorando el rendimiento general del programa.

6. Comparación: Syscalls vs Librerías:

El proyecto compara dos métodos de copia de archivos:

1. Llamadas al sistema (syscalls): utilizan `open(), read() y write().`
2. Librerías estándar: utilizan `fopen(), fread() y fwrite().`

Las funciones de librería suelen ser más eficientes para archivos pequeños debido al buffering interno que realizan en espacio de usuario. Sin embargo, las syscalls permiten mayor control y se aproximan más al funcionamiento interno del sistema operativo.

7. Manejo de errores:

El programa implementa manejo de errores robusto utilizando el “valor errno”. Esto permite detectar problemas como:

- Archivo inexistente
- Falta de permisos
- Errores de lectura o escritura

En caso de error, el programa informa al usuario y realiza hasta tres intentos antes de declarar un fallo definitivo.

```
PS C:\Users\Ismael\Documents\Repositories\Parcial-2---Sistemas-Operativos> ./smart_backup pruebaInexistente.txt copia.txt
BACKUP EAFITOS

Archivo proporcionado por el usuario
Origen:  pruebaInexistente.txt
Destino: copia.txt

  [ERROR] No se encontro: pruebaInexistente.txt
  Asegurate de tener el archivo en la misma carpeta.
PS C:\Users\Ismael\Documents\Repositories\Parcial-2---Sistemas-Operativos>
```

8. Pruebas de rendimiento:

Para analizar el rendimiento del sistema, el programa mide el tiempo de ejecución de cada método de copia utilizando funciones de la librería `time.h`. Los resultados se presentan en una tabla comparativa que muestra el tiempo de ejecución y la velocidad de copia.

Para archivo de 1 KB:

```
==============================================================================
                          TABLA COMPARATIVA
==============================================================================
|        |   SYSCALL (llamada directa)   |     LIBRERIA (buffered)       |
| Tamano |    Tiempo     |   Velocidad   |    Tiempo     |   Velocidad   |
|--------|---------------|---------------|---------------|---------------|
| 0000000000 B |  0.000000 s  |      0.00 MB/s |  0.000000 s  |      0.00 MB/s |
|--------|---------------|---------------|---------------|---------------|
==============================================================================
```

Para archivo de 10 MB:

```
==============================================================================
                          TABLA COMPARATIVA
==============================================================================
|        |   SYSCALL (llamada directa)   |     LIBRERIA (buffered)       |
| Tamano |    Tiempo     |   Velocidad   |    Tiempo     |   Velocidad   |
|--------|---------------|---------------|---------------|---------------|
| 0000000010 MB |  0.044000 s  |    227.27 MB/s |  0.039000 s  |    256.41 MB/s |
|--------|---------------|---------------|---------------|---------------|
==============================================================================
```

9. Resultados esperados:

Se espera que el rendimiento de cada método varíe dependiendo del tamaño del archivo. En archivos pequeños, las librerías pueden ser más rápidas debido a su buffering interno. En archivos grandes, las syscalls pueden mostrar un rendimiento comparable o superior dependiendo de la implementación.

Prueba 1:

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS                                   powershell

PS C:\Users\Ismael\Documents\Repositories\Parcial-2---Sistemas-Operativos> echo "im a text archive used for the report" > pruebas/pruebareport.tx
t
PS C:\Users\Ismael\Documents\Repositories\Parcial-2---Sistemas-Operativos> gcc main.c backup_engine.c -o smart_backup
PS C:\Users\Ismael\Documents\Repositories\Parcial-2---Sistemas-Operativos> ./smart_backup pruebas/pruebareport.txt pruebas/copiareport.txt
BACKUP EAFITOS

Archivo proporcionado por el usuario
Origen:  pruebas/pruebareport.txt
Destino: pruebas/copiareport.txt

Ejecutando copia...


==========================================================================
                          TABLA COMPARATIVA
==========================================================================
|        |   SYSCALL (llamada directa)   |     LIBRERIA (buffered)       |
| Tamano |    Tiempo      |   Velocidad   |    Tiempo      |   Velocidad   |
|--------|---------------|---------------|---------------|---------------|
| 0000000000 B |  0.000000 s  |      0.00 MB/s |  0.000000 s  |      0.00 MB/s |
|--------|---------------|---------------|---------------|---------------|
==========================================================================
```

Prueba 2:

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS                                   powershell

PS C:\Users\Ismael\Documents\Repositories\Parcial-2---Sistemas-Operativos> fsutil file createnew archivo10MB.txt 10485760
El archivo C:\Users\Ismael\Documents\Repositories\Parcial-2---Sistemas-Operativos\archivo10MB.txt está creado
PS C:\Users\Ismael\Documents\Repositories\Parcial-2---Sistemas-Operativos> ./smart_backup archivo10MB.txt copia10MB.txt
BACKUP EAFITOS

Archivo proporcionado por el usuario
Origen:  archivo10MB.txt
Destino: copia10MB.txt

Ejecutando copia...


==========================================================================
                          TABLA COMPARATIVA
==========================================================================
|        |   SYSCALL (llamada directa)   |     LIBRERIA (buffered)       |
| Tamano |    Tiempo      |   Velocidad   |    Tiempo      |   Velocidad   |
|--------|---------------|---------------|---------------|---------------|
| 0000000010 MB |  0.044000 s  |    227.27 MB/s |  0.039000 s  |    256.41 MB/s |
|--------|---------------|---------------|---------------|---------------|
==========================================================================
```

10.Conclusiones:

Este proyecto demuestra las diferencias entre operaciones realizadas en espacio de usuario y operaciones cercanas al kernel. Además, permite comprender cómo el manejo de buffers, llamadas al sistema y librerías influyen en el rendimiento de operaciones de entrada y salida en un sistema operativo.